JWT (JSON Web Token) - Complete Notes

1. Introduction to JWT

JWT (JSON Web Token) is a secure token-based authentication mechanism used to verify the identity of users in web and mobile applications.

After a user successfully logs in, the server generates a JWT and sends it to the client. The client stores this token and sends it with future requests to access protected resources.

JWT is commonly used in MERN Stack applications because it is stateless, scalable, and easy to integrate with APIs.

---

2. Why JWT is Needed

Without JWT:

- Users would need to authenticate repeatedly.
- The server would need to store session information for every user.
- Scaling applications becomes difficult.

With JWT:

- User logs in once.
- Server generates a token.
- Client stores the token.
- Client sends the token with every request.
- Server verifies the token and grants access.

---

3. JWT Authentication Flow

Step 1: User Login

The client sends login credentials to the server.

Example:

Email: user@gmail.com

Password: 123456

Step 2: Server Verifies Credentials

The server checks whether:

- The email exists.
- The password is correct.

If the credentials are valid, the server generates a JWT.

Step 3: Server Generates JWT

Example:

```
const token = jwt.sign( { id: user._id }, "secretKey" );
```

The generated token looks like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Step 4: Client Stores Token

The token can be stored in:

- Local Storage
- Session Storage
- HTTP-Only Cookies

Example:

```
localStorage.setItem("token", token);
```

Step 5: Client Sends Token

The token is sent with future requests.

Example:

```
Authorization: Bearer <token>
```

Step 6: Server Verifies Token

The server verifies the token.

Example:

jwt.verify(token, "secretKey");

If valid:

- Access Granted

If invalid:

- Access Denied

---

4. Structure of JWT

A JWT consists of three parts:

Header.Payload.Signature

Example:

xxxxx.yyyyy.zzzzz

---

5. Header

The Header contains metadata about the token.

Example:

{ "alg": "HS256", "typ": "JWT" }

Where:

alg = Signing Algorithm

typ = Token Type

---

6. Payload

The Payload contains user-related information.

Example:

{ "id": "123", "name": "Shahid", "role": "user" }

Important:

The payload is not encrypted.

Anyone can decode it.

Never store:

- Passwords
- OTPs
- Sensitive information

inside the payload.

---

7. Signature

The Signature is generated using:

- Header
- Payload
- Secret Key

Example:

jwt.sign(payload, "secretKey")

Purpose:

- Ensures data integrity.
- Prevents token tampering.
- Verifies authenticity.

---

8. Installing JWT

Install the jsonwebtoken package:

npm install jsonwebtoken

---

9. Creating a JWT

Example:

const jwt = require("jsonwebtoken");

const token = jwt.sign({ id: user._id }, "secretKey", { expiresIn: "1d" });

Explanation:

Payload: { id: user._id }

Secret Key: "secretKey"

Options: { expiresIn: "1d" }

The token will expire after one day.

---

10. Verifying JWT

Example:

const decoded = jwt.verify(token, "secretKey");

console.log(decoded.id);

The verify() method:

- Checks whether the token is valid.
- Decodes the payload.
- Returns stored data.

---

11. Authentication Middleware

Example:

const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {

```
const token = req.headers.authorization;
```

```
if (!token) {
    return res.status(401).json({
        message: "Token Missing"
    });
}

try {

    const decoded = jwt.verify(
        token,
        "secretKey"
    );

    req.user = decoded;

    next();

} catch (error) {

    return res.status(401).json({
        message: "Invalid Token"
    });

}
```

};

module.exports = authMiddleware;

Purpose:

- Checks whether the user is authenticated.
- Allows access to protected routes.

---

12. Protected Route Example

app.get("/profile", authMiddleware, (req, res) => {

```
    res.json({
        user: req.user
    });

}
```

);

Only authenticated users can access this route.

---

13. Advantages of JWT

1. Stateless Authentication
2. No Session Storage Required
3. Scalable Architecture
4. Suitable for APIs
5. Works with Mobile and Web Apps
6. Fast Verification Process

---

14. Disadvantages of JWT

1. Token cannot be modified after creation.
2. Larger payload increases token size.
3. If stolen, the token can be misused until expiration.
4. Requires secure storage.

---

15. JWT vs Session Authentication

JWT Authentication

- Token stored on client.
- Stateless.
- Better for APIs.
- Highly scalable.

Session Authentication

- Session stored on server.
- Stateful.
- Better for traditional applications.
- Requires session management.

---

16. Interview Questions

What is JWT?

JWT (JSON Web Token) is a secure token-based authentication mechanism used to verify user identity. After successful login, the server generates a signed token containing user information. The client stores this token and sends it with future requests. The server verifies the token before granting access to protected resources.

Why is JWT called Stateless?

JWT is called stateless because the server does not store session information. All required user information is stored inside the token itself.

What are the three parts of JWT?

1. Header
2. Payload
3. Signature

Which package is used to implement JWT in Node.js?

jsonwebtoken

Which methods are commonly used?

Creating Token:

jwt.sign()

Verifying Token:

jwt.verify()

---

17. Quick Revision

User Login ↓ Server Verifies Credentials ↓ JWT Generated ↓ Client Stores Token ↓ Client Sends Token ↓ Server Verifies Token ↓ Access Granted

Formula:

Login → Generate Token → Store Token → Send Token → Verify Token → Access Protected Routes